Fourth Down Decision Model

Alejandro Zamudio

2026-05-23

Over the past few seasons, the decision to keep the offense on the field and "go for it" on 4th down has become much more popular. What used to be seen as a risky and desperate choice for most offenses has shifted to a mindset of utilizing the extra down in manageable situations. According to a 2025 article by Fox Sports, teams went for it on 4th down 60% more times in the 2024 season compared to the decade before. The decision to keep the offense on the field has shifted vastly, however this decision comes down to a multitude of factors. Score differential, situation of the game, time remaining and distance from the 1st down and the teams own mentality on 4th down, many factors play into whether teams are willing to take the safe option or take the risk to extend their drive.

In this project, I aim to create three different Machine Learning models to predict the likelihood of converting on fourth down given various "pre-snap" features available. Using a Logistic Regression as my primary model, I will test different features to evaluate their performance and effectiveness within the model, ultimately selecting the set that appears ideal for the task. Features will be purely "pre-snap", and will not include any stat or variable that is available or computed after a play ends. Once creating my primary model, I will also train a Random Forest and XGBoost with the intent of comparing its performance to the Logistic Regression. With these two types of models performing well on non-linear data, I am interested in how similar or different the predicted probabilities are from the primary model's prediction.

The goal of the project is not to select the best model, but highlight differences in the three models predictions. The Logistic Regression will serve as my primary, and I will compare results of their predictions based on scenarios provided to the models. As part of the process, I will be choosing a probability threshold to base my classifications of decision off of. This threshold will be the difference between the decision to "Kick the ball" (whether punt of try a field goal) or "Go For It" (run a play on 4th down).

Data

```
# Create Dataset utilized nflfastR for play by play
NFL_pbp <- load_pbp(2021:2025)
```

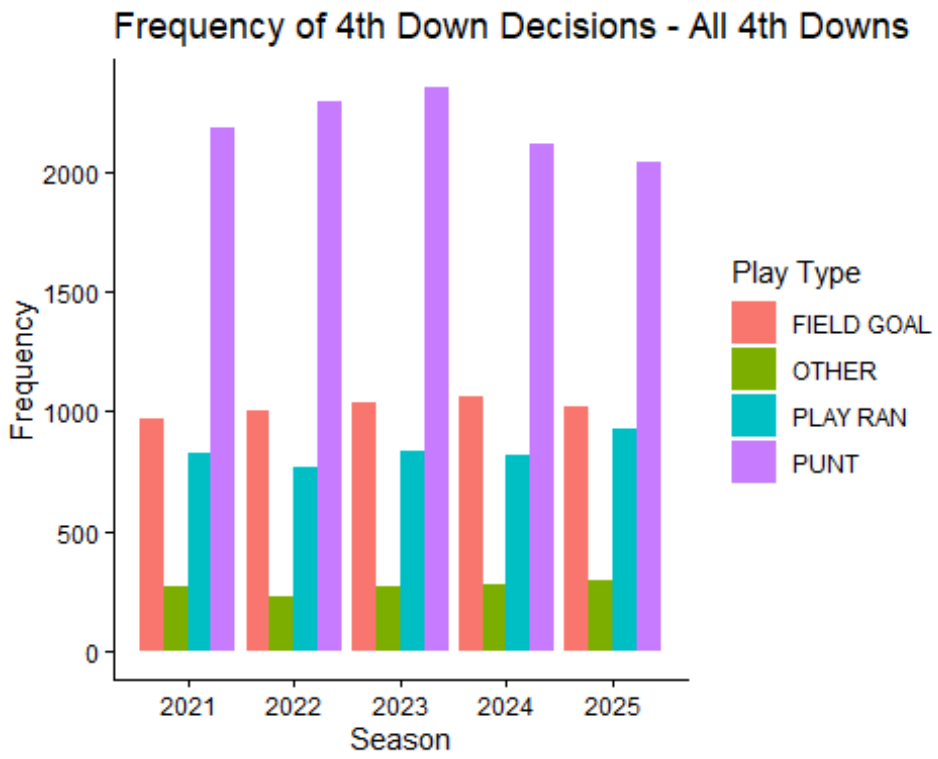
The project will use play by play data from nflfastR, an NFL based package in R (library call in the markdown file). I will utilize data from the past 5 seasons for both an initial analysis for 4th down trends and in training and testing my model. For all models, training sets will be use the 2021-2023 seasons, validation sets will use the 2024 season, and testing sets

will use the 2025 season. All final versions of each model will be trained on the entire dataset.

Analysis of Fourth Downs

To begin the project, I start off by looking at the prevalence of 4th down attempts in the past few seasons. The first chart I show are the frequency of particular 4th Down choices such as running a play, punting, or kicking a field goal. Note that "Other" in the dataset refers to plays where a flag was thrown or where a Quarterback kneeled on the play.

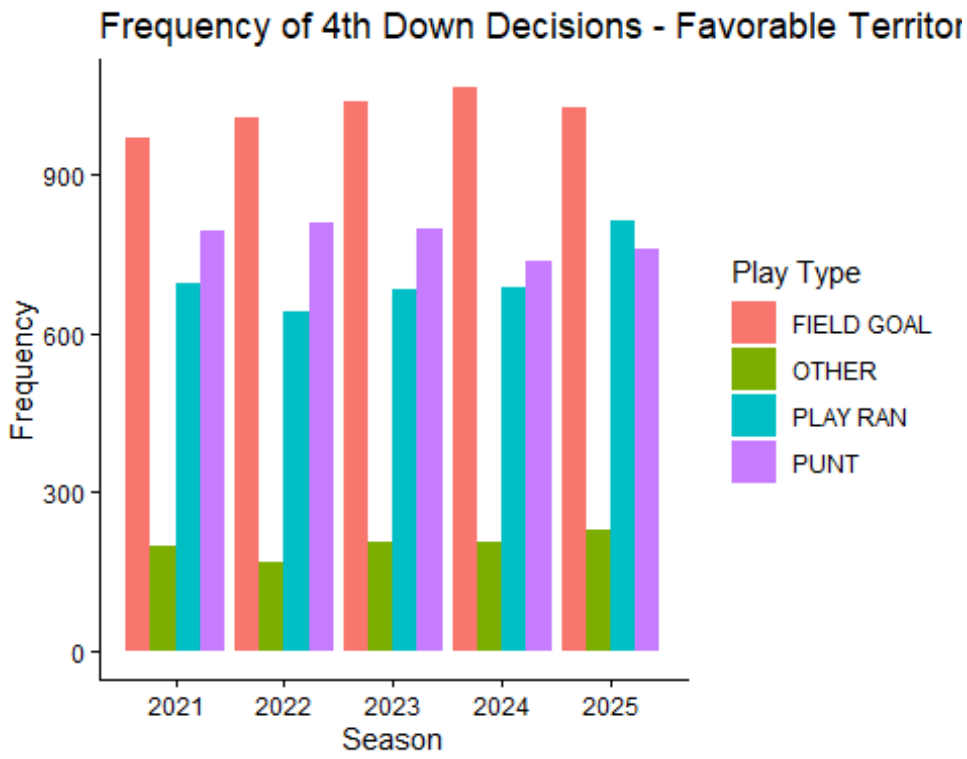
```
# Get all plays on 4th Down
FourthDowns <- NFL_pbp %>% filter(down == 4)

# Groupings for OTHER and PLAY RAN
OtherPlays <- c(NA, "qb_kneel", "no_play")
RanPlay <- c("pass", "run")

# Create additional column for modified play type
FourthDowns <- FourthDowns %>% mutate(FD_type_play = case_when(
  play_type == 'punt' ~ "PUNT",
  play_type == 'field_goal' ~ "FIELD GOAL",
  play_type %in% RanPlay ~ "PLAY RAN",
  play_type %in% OtherPlays ~ "OTHER"
))

# Create df from group by function
FD_play_dist <- FourthDowns %>% group_by(season, FD_type_play) %>%
summarise(Count = n())

## `summarise()` has grouped output by 'season'. You can override using the
## `.groups` argument.

ggplot(FD_play_dist, aes(fill=FD_type_play, y=Count, x=season)) +
geom_bar(position="dodge", stat="identity") + xlab("Season") +
ylab("Frequency") + labs(title = "Frequency of 4th Down Decisions - All 4th
Downs", fill = "Play Type") + theme_classic()
```



From an initial look at the play-by-play data over the past five seasons, punting is the primary choice among the league on 4th down. One thing to note is while Field Goals have remained consistent, 2025 shows a notable increase in plays being ran with the other 4 seasons fluctuating.

With most choices to go for it on 4th down occurring in field positions closer to mid field or beyond into opponent territory, I also wanted to look at the distribution in favorable territory. The next graph looks at the frequency of 4th down choices on plays from the possession team's own 40 and beyond.

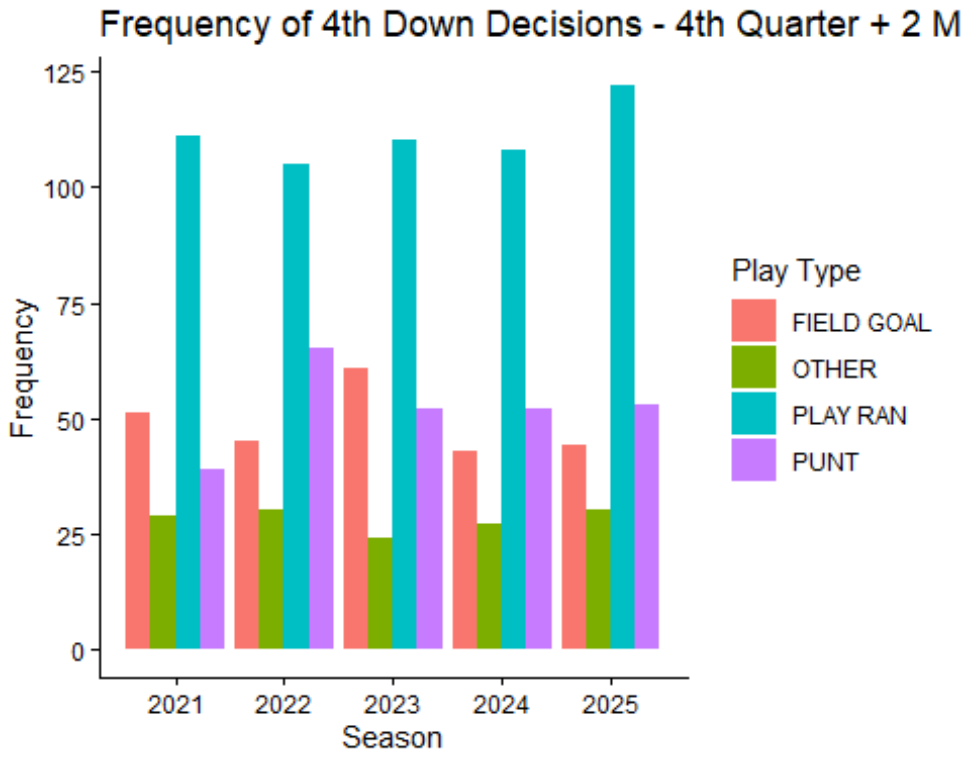
```
# Filter to only include situations where ball is on the offense own 40 or
beyond
FD_Probable <- FourthDowns %>% filter(yardline_100 <= 60)

# Find distributon of play types for plays 4th downs on the 40 and beyond
FD_prob_play_dist <- FD_Probable %>% group_by(season, FD_type_play) %>%
summarise(Count = n())

## `summarise()` has grouped output by 'season'. You can override using the
## `.groups` argument.

ggplot(FD_prob_play_dist, aes(fill=FD_type_play, y=Count, x=season)) +
geom_bar(position="dodge", stat="identity") +  xlab("Season") +
ylab("Frequency") + labs(title = "Frequency of 4th Down Decisions - Favorable
Territory", fill = "Play Type") + theme_classic()
```



The graph shows a similar trend from the previous in that Field Goals have remained consistent as a choice on 4th down. Furthermore, the choice to go for it has seen fluctuation with 2025 seeing the highest occurrences.

Lastly, with the choice to go for it often occurring late in the game with situations to stay alive or end the game, I wanted to look at the distribution in the 4th quarter with 2 minutes remaining in the game. The following chart only factors in 4th down plays in the 4th quarter where the time left in the game is 120 minutes or less

```
# Filter to only include situations where game is in the 4th quarter and with
120 seconds or less remaining
FD_FQ <- FourthDowns %>% filter(qtr <= 4, game_seconds_remaining < 120)

# Find distribution of play types for plays 4th downs in the 4th and with 120
secs or less remaining
FD_FQ_play_dist <- FD_FQ %>% group_by(season, FD_type_play) %>%
summarise(Count = n())

## `summarise()` has grouped output by 'season'. You can override using the
## `.groups` argument.

ggplot(FD_FQ_play_dist, aes(fill=FD_type_play, y=Count, x=season)) +
geom_bar(position="dodge", stat="identity") + xlab("Season") +
ylab("Frequency") + labs(title = "Frequency of 4th Down Decisions - 4th
Quarter + 2 Mins Left", fill = "Play Type") + theme_classic()
```



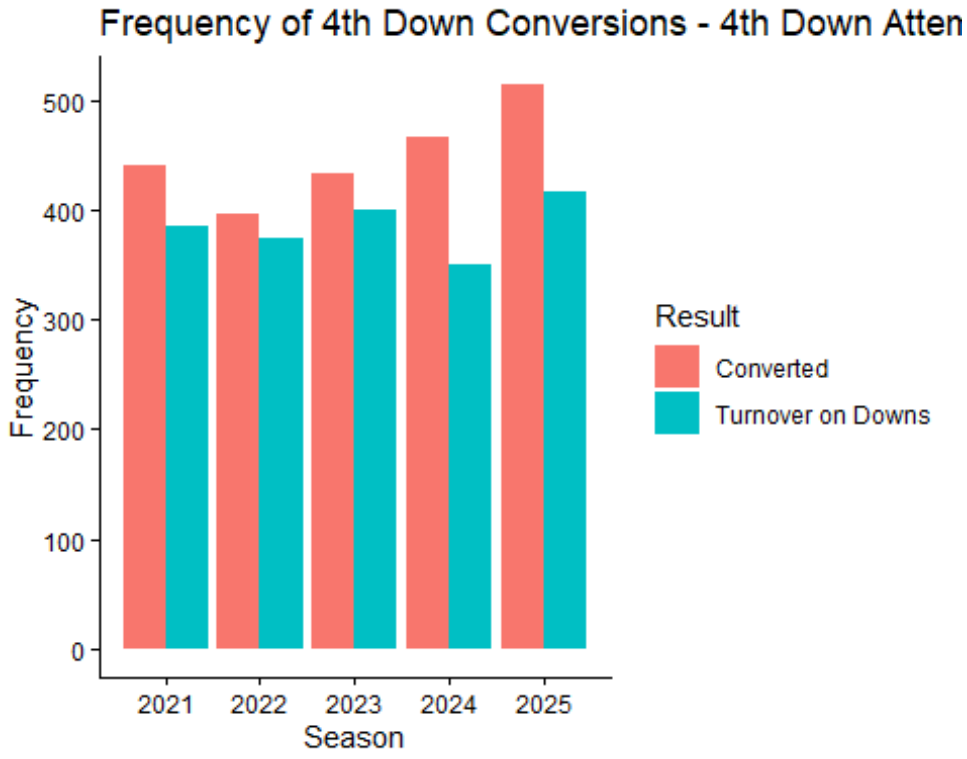
The graph shows that in these late game scenarios; teams very often opt to go for it. However, its important to note this graph does not factor in position on the field, score differential and other situation factors that play into these decisions. However, we do see the trend that late in the game, teams are very likely to go for it.

Conversion Rates

I wanted to look at the percentage of plays that are converted on 4th down. The following graph and chart will be limited to only instances where a play was ran; not including fake punts, fake field goals or other instances other than a run or pass play.

```
# Find instances in which a play was ran on fourth down, create variable for converted
Fourth_Conversions <- FourthDowns %>% filter(FD_type_play == "PLAY RAN") %>% mutate(
  Converted_fourth = case_when(
    fourth_down_converted == 1 ~ 'Converted',
    fourth_down_converted == 0 ~ "Turnover on Downs"
  )
)

# Find ditribution of conversions based on season
FD_Conversions <- Fourth_Conversions %>% group_by(season, Converted_fourth) %>% summarise(Count = n())
```

```
## `summarise()` has grouped output by 'season'. You can override using the
## `.groups` argument.

ggplot(FD_Conversions, aes(fill=Converted_fourth, y=Count, x=season)) +
geom_bar(position="dodge", stat="identity") + xlab("Season") +
ylab("Frequency") + labs(title = "Frequency of 4th Down Conversions - 4th
Down Attempts", fill = "Result") + theme_classic()
```



As noted in our analysis of 4th Down plays, each season saw slight increase in the number of times teams went for it on 4th down. This bar chart shows that over each seasons, teams are progressively converting more 4th downs in relation to the amount of times they are going for it. The following table shows the conversion rate for all five seasons, showing that the conversion percentages seem to fall close to 50%.

```
Fourth_Conversions %>% group_by(season) %>% summarise(`Plays Ran` = n(),
`Conversion Rate` = sum(fourth_down_converted) / n())
```

```
## # A tibble: 5 × 3
##   season `Plays Ran` `Conversion Rate`
##    <int>       <int>             <dbl>
## 1   2021         826             0.534
## 2   2022         770             0.514
## 3   2023         833             0.520
## 4   2024         816             0.571
## 5   2025         931             0.552
```

Model Creation

I will train three different ML models with the goal of generating a probability of converting on 4th down given certain pre-snap features. The idea is to have model that informs on the likelihood of converting a 4th down play given the scenario and positioning on the field. The features are intended to be metrics and variables that are known before a play is ran, serving as a tool to making the decision on whether to try to run a play on 4th down.

The Logistic Regression will serve as the primary model of choice. I will be doing my feature selection and main decision making using this model, while also using the same features for the other two models. I will highlight the predicted probability of conversion compared to actual results, and comparing the predictions of all three models on different scenarios.

Logistic Regression

The purpose of the Logistic Regression will be to generate a probability of getting the first down given the chosen pre-snap features given to the model. The data used will simply be plays in which teams ran an actual play (pass or run) on fourth down. This will exclude plays involving punts, field goals, QB Kneels, and plays where a flag affected influenced yards gained or lost. With this being my primary model, most of the training, feature selection, validation and testing was performed on this model to ensure it was performing as optimal as possible.

Target Variable: fourth_down_converted - This is a Boolean variable that indicates whether a team successfully gained a 1st down on a 4th down play. This will serve as the primary target variable for all three models. - With our data only including plays where a play was ran on 4th down, all rows have an instance for this variable and no missing values had to be inputted.

Predictor Variables:

- **yardline_100 (Yard Line):** Numeric Variable indicating the yard line in which the ball has been placed for the play. With 50 representing midfield (50 yard line), numbers between 50 and 100 represent plays in the possession team's own territory, and number between 0 and 50 represent plays in the defensive team's territory –
- **ydstogo (Yards to Go):** Numeric Variable indicating how many yards are left to get the 1st down. Example: instance with a 5 in the column indicate 4th and 5 (4th down since all the plays are 4th down plays).
- **goal_to_go (Goal to Go):** Boolean Variable indicating whether the possession team's is in a goal down situation. This would represent 4th and Goal in terms of the current model
- **score_differential:** Numeric Variable representing the difference between the possession team's score and the defensive team's score. A positive number means

the possession team is leading while a negative number means the defensive team is leading.

- **half_seconds_remaining:** Numeric Variable representing the total amount of time remaining in seconds remaining in the current half. Numbers are similar for the same time remaining in both the 1st half and 2nd half. The decision to use this variable over seconds left in the game was based on belief that game seconds could heavily influence prediction.
- **td_prob (Touchdown Probability):** Numeric Variable representing the probability that the possession team scores a touchdown. The probability is calculated before the play is ran, serving as pre-snap information. While this probability is based on factors such as field position, I felt as the metric is crucial towards the decision to go for it. Variable was checked to ensure it was not too heavily correlated with other predictors in the model.
- **matchup_strength:** Numeric Variable representing the cumulative EPA on 4th down gained by the possession team and conceded by the defensive team. This variable was engineered to account for the strengths of both the offense and defense. A positive number favors the offense while a negative number favors the defense. Adding the two EPA totals, the number for each team's offense and defense is updated on a game basis and is reset for each season (EPA on 4th down will always be 0 in first games of the season). The creation of the variable is shown below.

```
# Possesion Team EPA on 4th Downs
games <- Fourth_Conversions %>%
  filter(!is.na(epa)) %>%
  group_by(season, game_id, posteam) %>%
  summarise(
    game_epa = mean(epa),
    .groups = "drop"
  ) %>%
  arrange(posteam, season, game_id) %>%
  group_by(posteam, season) %>%
  mutate(
    offense_epa_season = lag(cummean(game_epa))
  ) %>%
  ungroup()

# Defensive Teams EPA on 4th Downs
def_games <- Fourth_Conversions %>%
  filter(!is.na(epa)) %>%
  group_by(season, game_id, defteam) %>%
  summarise(
    game_def_epa = mean(epa),
    .groups = "drop"
  ) %>%
```

```
  arrange(defteam, season, game_id) %>%
  group_by(defteam, season) %>%
  mutate(
    defense_epa_season = lag(cummean(game_def_epa))
  ) %>%
  ungroup()

# Join possession team EPA totals on rows
Fourth_Conversions <- Fourth_Conversions %>%
  left_join(
    games %>%
      select(season, game_id, posteam, offense_epa_season),
    by = c("season", "game_id", "posteam")
  )

# Join defensive team EPA totals on rows
Fourth_Conversions <- Fourth_Conversions %>%
  left_join(
    def_games %>%
      select(season, game_id, defteam, defense_epa_season),
    by = c("season", "game_id", "defteam")
  )

# Convert NA's for all first games into 0
Fourth_Conversions <- Fourth_Conversions %>%
  mutate(
    offense_epa_season = ifelse(is.na(offense_epa_season), 0,
offense_epa_season),
    defense_epa_season = ifelse(is.na(defense_epa_season), 0,
defense_epa_season)
  )

# Create matchup strength
Fourth_Conversions <- Fourth_Conversions %>%
  mutate(
    matchup_strength = offense_epa_season + defense_epa_season
  )
```

Before reaching my final set of features for the model, I tested multiple logistic regressions with various combinations of variables in the final set. All versions of the logistic regression were trained on the 2021-2023 data and validated using the 2024 data. To evaluate the models, I frequently looked at AUC and Log Loss as a representation of how well they were performing. To do this, I used a probability threshold of 0.55 to separate the predicted probabilities by each model into binary decisions of 0 representing “Kick” (Less than .55) and 1 representing “Go For It” (0.55 or Greater). During this process, all of the models maintained a AUC between 0.63 and 0.68 and a log loss that ranged from 0.63 to 0.88. Each model saw one evaluation metric increase while the other decreased. I tried versions with few variables such as only with “yardline_100” and seconds left in half, used different

variations of variables such as choosing between game, half and quarter seconds, and tried changing certain variables to categorical to see what changes occur. The final set of features were the set that produced the most ideal combination of AUC and Log Loss

This 2025 season data was saved for testing data once finalizing the set of features I wanted to use for the final model. Along with AUC and Log Loss, I also looked at the confusion matrix for the classifications using the model. As mentioned, the binary classification was made using a threshold of 0.55. However, the interpretation of the matrix are that true positive and negatives represent situations where a successful 4th down conversion occurred with a “Go For It” recommendation and a play where a 4th down was not converted occurred with a “Kick” recommendation. While I did aim to find a model and threshold that could give more true positive and negatives, I understood that sometimes scenarios can favor each team in which a low probability of conversion can still be converted and a high probability of conversion is not.

The following code is the final model:

```
# Create Final Model
FINAL_LogReg_FDC <- glm(fourth_down_converted ~ yardline_100 + ydstogo +
goal_to_go + score_differential + half_seconds_remaining + td_prob +
matchup_strength, data = Fourth_Conversions, family = "binomial")

# Evaluate the model
summary(FINAL_LogReg_FDC)

##
## Call:
## glm(formula = fourth_down_converted ~ yardline_100 + ydstogo +
##     goal_to_go + score_differential + half_seconds_remaining +
##     td_prob + matchup_strength, family = "binomial", data =
Fourth_Conversions)
##
## Coefficients:
##                           Estimate Std. Error z value Pr(>|z|)
## (Intercept)             4.601e-01  1.202e-01   3.827  0.00013 ***
## yardline_100            3.017e-03  1.827e-03   1.651  0.09872 .
## ydstogo                -1.576e-01  1.156e-02 -13.627  < 2e-16 ***
## goal_to_go             -2.987e-01  1.320e-01  -2.264  0.02359 *
## score_differential      3.593e-03  2.963e-03   1.213  0.22530
## half_seconds_remaining  1.039e-04  7.372e-05   1.410  0.15863
## td_prob                 7.111e-01  3.175e-01   2.240  0.02509 *
## matchup_strength       -4.599e-02  1.948e-02  -2.361  0.01825 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 5764.0  on 4175  degrees of freedom
## Residual deviance: 5319.8  on 4168  degrees of freedom
```

```
## AIC: 5335.8
##
## Number of Fisher Scoring iterations: 4

# Display the AUC, Log Loss and Confusion Matrix
evaluate_model(FINAL_LogReg_FDC, 0.55)

## Setting levels: control = 0, case = 1

## Setting direction: controls < cases

## $confusion_matrix
##           Actual
## Predicted   0   1
##         0 225 156
##         1 192 358
##
## $auc
## Area under the curve: 0.6606
##
## $log_loss
## [1] 0.6479071
```

In the final model, yards to go has the highest significance out of all the features within the model, with match-up strength, touch down probability and score differential following behind it. When put into context of the game, these are real factors that do have heavy implications on the decision to go for it or kick the ball. Our confusion matrix shows a good amount of cases where predicted go for it scenarios were converted and predicted kick scenarios were not. Lastly, our AUC and Log Loss are fairly good predictive power of the model on likelihood of converting on 4th.

Random Forest

With many of the features in our dataset being non linear, I was also interested in training a Random Forest to see if it can make improvement to the predictive power, along with seeing what features it sees as the most important. As mentioned, I will be using the same set of features used for the Logistic Regression for consistency. This will allow me to compare the predictions given by the trained models. In terms of training the model, I use the “Ranger” package to create the model.

Similar to the Logistic Regression, another method is created evaluate the models confusion matrix, AUC and Log Loss. I also create another method to look at feature importance and prediction error. This is simply a matter of seeing what the model deems as important to its prediction and how far off the prediction are from our thresholds classification of the decision.

```
# For this model, we will train on the 2021-2024 season. Testing will occur
on the 2025 season
training_data <- Fourth_Conversions %>% filter(season <= 2024)
```

```r
# Method Evaluates Confusion Matrix, AUC and Log Loss of model, takes in
Random Forest and threshold probability
evaluate_rf_model <- function(model, threshold = 0.50) {

  # All data tested on the 2025 season
  testing_data <- Fourth_Conversions %>% filter(season == 2025)

  # Generate prediction probabilities and create classification decision
  probs <- predict(model, testing_data)$predictions[,2]
  preds <- ifelse(probs > threshold, 1, 0)

  # Generate Confusion Matrix
  cm <- table(Predicted = preds, Actual = testing_data$fourth_down_converted)

  # Generate AUC
  auc <- pROC::auc(testing_data$fourth_down_converted, probs)

  # Find Log Loss
  eps <- 1e-15
  probs_clipped <- pmin(pmax(probs, eps), 1 - eps)
  logloss <- -mean(
    testing_data$fourth_down_converted * log(probs_clipped) +
    (1 - testing_data$fourth_down_converted) * log(1 - probs_clipped)
  )

  # Present Confusion Matrix, AUC and Log Loss
  list(
    confusion_matrix = cm,
    auc = auc,
    log_loss = logloss
  )
}

# Method to evaluate important features and prediction error
rf_summary <- function(model) {
  list(
    importance = sort(model$variable.importance, decreasing = TRUE),
    oob_error = model$prediction.error
  )
}
```

When trying the initial version of the random forest, the AUC was 0.639 while the log loss was at 0.875, an slightly worst performance from the Logistic Regression. Using the 0.55 threshold for classification, the model struggled with low probability kick recommendations that were converted for 1st downs. With this in mind, I wanted to see if I cound optimizr the prediction power by performing cross validation to see if I could improve the hyper parameters. In terms of generating the best AUC, the combination of 2,

20 and 500 (mtry, min.node.size, num.trees) appears to be our best set while for our lowest log loss, 2, 20 and 500 appears to be our best set. With a small difference in the two between the sets of parameters, I will utilize the set with the best log loss for the final model

```
# Create the final model
rf_model_FINAL <- ranger(fourth_down_converted ~ yardline_100 + ydstogo +
goal_to_go + score_differential + half_seconds_remaining + td_prob +
matchup_strength, data = Fourth_Conversions,

  probability = TRUE,
  mtry = 2,
  min.node.size = 20,
  num.trees = 500,
  importance = "impurity",
  seed = 27
)

rf_summary(rf_model_FINAL)

## $importance
##                td_prob half_seconds_remaining       matchup_strength
##             234.044137             199.233003             192.560012
##                ydstogo           yardline_100     score_differential
##             186.302844             151.257366             141.384603
##             goal_to_go
##               7.629765
##
## $oob_error
## [1] 0.2311321
```

In terms of the final model, touchdown probability, seconds remaining in the half and match-up strength we seen as the most important features in the model. Differing heavily from the Logistic Regression, the model seems to produce probability of conversions much closer to 50% and seems to produce recommendations to go for it when basing the prediction of the threshold chosen.

XGBoost

The last model I had interest in trying was an XGBoost. With the model aimed at iteratively building trees designed to correct prior prediction errors, I am interested to see how its predictions differ from the Logistic Regression and Random Forest. The intent is to see if it can improve upon the classification of decisions as well.

The initial trained XGBoost showed a fairly similar performance in terms of Log Loss and AUC compared to both the Logistic Regression and Random Forest. However, the confusion matrix for the model was much closer to the results of the Logistic Regression in that it produced more true positive and negatives than false positive and negatives (using the 0.55 threshold for classification). Looking at the predictions themselves, the generated

probabilities did seem to be higher than those by the Random Forest, however still fairly more conservative compared to the Logistic Regression.

```
features <- c(
  "yardline_100",
  "ydstogo",
  "goal_to_go",
  "score_differential",
  "half_seconds_remaining",
  "td_prob",
  "matchup_strength"
)

X <- as.matrix(Fourth_Conversions[, features])
y <- Fourth_Conversions$fourth_down_converted

dtrain <- xgb.DMatrix(data = X, label = y)

final_XGB <- xgb.train(
  data = dtrain,
  objective = "binary:logistic",
  nrounds = 100,
  max_depth = 3,
  eta = 0.1,
  eval_metric = "logloss",
  verbose = 0
)
```

```
## Warning in check.deprecation(deprecated_train_params, match.call(), ...):
## Passed invalid function arguments: max_depth, eta, eval_metric. These
should be
## passed as a list to argument 'params'. Conversion from argument to
'params'
## entry will be done automatically, but this behavior will become an error
in a
## future version.
```

```
## Warning in check.custom.obj(params, objective): Argument 'objective' is
only
## for custom objectives. For built-in objectives, pass the objective under
## 'params'. This warning will become an error in a future version.
```

Scenario Testing

In this section, I will provide three different scenarios and evaluate the three different probabilities provided by each model. To remain consistent with the steps taken in the model creation, we will use a threshold of 0.55 as the difference between a “Go For It” decision (higher than 0.55) and “Kick” decision (lower than 0.55).

Scenario 1: Here are the variables for the first scenario

- Yard Line: 60 (Own 40)
- Yards to Go: 2
- Goal To Go: 0 (False)
- Score Differential: 1
- Half Seconds Remaining: 480 (8 Mins)
- TD Probability: 0.5
- Match Up Strength: -0.001

```
# Scenario 1 Data
scenario1 <- data.frame(yardline_100 = 60, ydstogo = 2, goal_to_go = 0,
score_differential = 1, half_seconds_remaining = 480,  td_prob = 0.5,
matchup_strength = -0.001)

# Feature order for XGBoost
features <- c("yardline_100", "ydstogo", "goal_to_go", "score_differential",
"half_seconds_remaining", "td_prob", "matchup_strength")

# Logistic Regression Prediction
lr_prob <- predict(FINAL_LogReg_FDC, newdata = scenario1, type = "response")

# Random Forest Prediction
rf_prob <- predict(rf_model_FINAL,data = scenario1)$predictions[,2]

# XGBoost Prediction
dscenario1 <- xgb.DMatrix(data = as.matrix(scenario1[, features]))

xgb_prob <- predict( final_XGB, dscenario1)

# Print Results
cat("Logistic Regression Probability:", round(lr_prob, 3), "\n")

## Logistic Regression Probability: 0.676

cat("Random Forest Probability:", round(rf_prob, 3), "\n")

## Random Forest Probability: 0.515

cat("XGBoost Probability:", round(xgb_prob, 3), "\n")

## XGBoost Probability: 0.561
```

The highest predicted probability of converting on 4th down in this scenario is from our Logistic Regression at 0.676, followed by the XGBoost at 0.561 and the Random Forest at

0.515. If using our threshold of 0.55, the Logistic Regression and XGBoost would recommend going for it.

Comments: The Logistic Regression provides a fairly high probability of conversion while the XGBoost's prediction is very close to the threshold for a first down. The scenario provided does present the question of does the possession team try to convert for fourth down to keep thr drive going in promising territory, or do they settle for a punt to avoid giving the opposing team the ball back in crucial territory and only up by 1.

Scenario 2: Here are the variables for the second scenario

- Yard Line: 4 (Opp 4)
- Yards to Go: 4
- Goal To Go: 1 (True)
- Score Differential: -7
- Half Seconds Remaining: 20 (20 Seconds)
- TD Probability: 0.6
- Match Up Strength: -2.0

```
# Scenario 2 Data
scenario2 <- data.frame(yardline_100 = 4, ydstogo = 4, goal_to_go = 1,
score_differential = -7, half_seconds_remaining = 20,  td_prob = 0.6,
matchup_strength = -2.0)

# Feature order for XGBoost
features <- c("yardline_100", "ydstogo", "goal_to_go", "score_differential",
"half_seconds_remaining", "td_prob", "matchup_strength")

# Logistic Regression Prediction
lr_prob <- predict(FINAL_LogReg_FDC, newdata = scenario2, type = "response")

# Random Forest Prediction
rf_prob <- predict(rf_model_FINAL,data = scenario2)$predictions[,2]

# XGBoost Prediction
dscenario2 <- xgb.DMatrix(data = as.matrix(scenario2[, features]))

xgb_prob <- predict( final_XGB, dscenario2)

# Print Results
cat("Logistic Regression Probability:", round(lr_prob, 3), "\n")
```

```
## Logistic Regression Probability: 0.51

cat("Random Forest Probability:", round(rf_prob, 3), "\n")

## Random Forest Probability: 0.45

cat("XGBoost Probability:", round(xgb_prob, 3), "\n")

## XGBoost Probability: 0.506
```

The highest predicted probability of converting on 4th down in this scenario is from our Logistic Regression at 0.51, followed by the XGBoost at 0.506 and the Random Forest at 0.45. If using our threshold of 0.55, all three models would recommend kicking the ball.

Comments: In a must score scenario, all the models recommend kicking the field goal. However with only 20 seconds remaining in the half and being down 7, the choice to go for it would almost always happen whether this was the end of the 1st half or end of the game. With a match up score of -2 possibly playing into the model's lower probability, it is interesting that all three give low probability of conversion.

Scenario 3: Here are the variables for the third scenario

- Yard Line: 80 (Own 20)
- Yards to Go: 1
- Goal To Go: 0 (True)
- Score Differential: 0
- Half Seconds Remaining: 300 (5 minutes)
- TD Probability: 0.2
- Match Up Strength: 4.1

```
# Scenario 3 Data
scenario3 <- data.frame(yardline_100 = 80, ydstogo = 1, goal_to_go = 0,
score_differential = 0, half_seconds_remaining = 300,  td_prob = 0.2,
matchup_strength = 4.1)

# Feature order for XGBoost
features <- c("yardline_100", "ydstogo", "goal_to_go", "score_differential",
"half_seconds_remaining", "td_prob", "matchup_strength")

# Logistic Regression Prediction
lr_prob <- predict(FINAL_LogReg_FDC, newdata = scenario3, type = "response")

# Random Forest Prediction
```

```
rf_prob <- predict(rf_model_FINAL,data = scenario3)$predictions[,2]

# XGBoost Prediction
dscenario3 <- xgb.DMatrix(data = as.matrix(scenario3[, features]))

xgb_prob <- predict( final_XGB, dscenario3)

# Print Results
cat("Logistic Regression Probability:", round(lr_prob, 3), "\n")

## Logistic Regression Probability: 0.629

cat("Random Forest Probability:", round(rf_prob, 3), "\n")

## Random Forest Probability: 0.432

cat("XGBoost Probability:", round(xgb_prob, 3), "\n")

## XGBoost Probability: 0.669
```

The highest predicted probability of converting on 4th down in this scenario is from the XGBoost at 0.669, followed by the Logistic Regression at 0.629 and the Random Forest at 0.45. If using our threshold of 0.55, the Logistic Regression and XGBoost would recommend going for it.

Comments: The Logistic Regression and XGBoost provide fairly decent probabilities of converting on fourth down in very risky scenarios. Although having a yard to reach the first down and having a huge match up difference, the field position still poses a high risk as a turnover put the opposing team in the red zone immediately. Other context about opponents and success by the possession team would be important before deciding to go for it.

Concluding Thoughts

When comparing all three models, it appears the primary model chosen (Logistic Regression) tends to predict higher probabilities in scenarios where yards to go favors the possession team and where the strength match up also favors the offense. When going off our 0.55 threshold for classification of a kick or go for it decision, the model tends to provide conversion probabilities that might be viewed as fairly ambitious (as seen in scenario 3). For the Random Forest, the model is heavily influenced by touchdown probability, match up strength and time remaining in the half. It serves as a more conservative prediction, rarely providing a probability above the chosen threshold to “go for it”. For the XGBoost, the model provided a high probability of conversion in the scenario with a more favorable match up strength, being heavily influenced by it. In this case, it can be a good medium between the Logistic Regression and Random Forest, or more ambitious.

While the models are meant to take in values available pre-snap (before the play begins), there is much room for improvement.

1. Additional or more analytically backed offensive and defensive strength metrics could serve as a better indicator or strength as compared to the one used in all three models. As mentioned, the intent of using EPA on 4th downs was to provide an understanding of how good offenses are in 4th downs for that season as well as knowing how good defenses are on limiting positive EPA on 4th downs. However a more reliable metric could serve as an additional feature of supplement for the chosen metric in this project.
2. The addition of player stats, both on the offensive and defensive side, could provide player context to the probability of conversion. While there is no guarantee adding specific metrics or stats heavily alter the models' predictions, it adds the player related context to the decision process.
3. The threshold for a kick and go for its decision can be improved to be based on a probability that is reflective of how aggressive teams are willing to be on 4th down. My choice of 0.55 was based on how well it classified true positive and true negatives, however different teams have different philosophies of how aggressive they want to be. Having a threshold reflective of how certain a team needs to be before taking the risk would be effective in how a decision is made.

Shiny App

For this project, a shiny app has been created to generate probabilities given variables provided by the user. The app is available to view and use in the github project folder associated for this project. The following is the link to the app:

https://019e8a4b-6786-3a48-10bb-f297f17abb36.share.connect.posit.cloud/